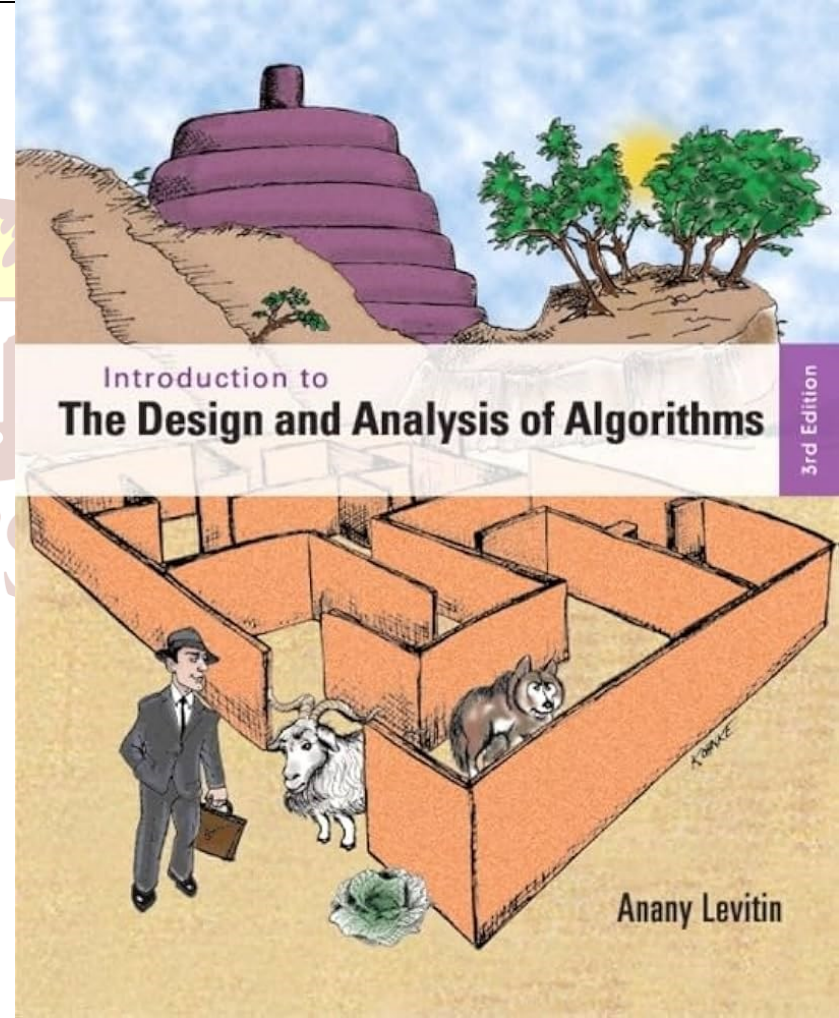


CS 07340 DESIGN AND ANALYSIS OF ALGORITHMS

WEEK 6 OVERVIEW

- Transform and Conquer
 - Balanced Search Trees
 - AVL Trees
 - 2-3 Trees

Rowan
University



TRANSFORM AND CONQUER

The next group of techniques is based on the idea of transforming the problem to an equivalent, yet simpler or more convenient form, solving that, and then translating it back into original form. There are three variations to consider:

- Instance Simplification
 - Transformation to a simpler instance of the same problem.
- Change of Representation
 - Transformation to a different representation.
- Problem Reduction
 - Transformation to a different problem for which an algorithm is already available.

PRESORTING

Presorting is sorting! We have algorithms (*MergeSort*, later we will see *HeapSort*) that have complexity $O(n \log(n))$. At the end of the semester, we will see that a **Decision Trees** provide a proof that comparison-based sorting is $\Omega(n \log(n))$ (theoretical lower bound), and hence $\Theta(n \log(n))$ (as we have an upper bound from actual algorithms).

The idea behind presorting is that some algorithms are much faster on sorted data, so it is worthwhile to sort data before applying these algorithms, especially if we plan on applying some algorithms repeatedly.

An example is the [element uniqueness problem](#), related to finding repeated elements and the [collision problem](#).

ALGORITHM *PresortElementUniqueness*($A[0..n - 1]$)

//Solves the element uniqueness problem by sorting the array first

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Returns “true” if A has no equal elements, “false” otherwise

sort the array A

for $i \leftarrow 0$ **to** $n - 2$ **do**

if $A[i] = A[i + 1]$ **return false**

return true

ALGORITHM *PresortMode*($A[0..n - 1]$)

//Computes the mode of an array by sorting it first

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: The array's mode

sort the array A

$i \leftarrow 0$ //current run begins at position i

modefrequency $\leftarrow 0$ //highest frequency seen so far

while $i \leq n - 1$ **do**

$runlength \leftarrow 1$; $runvalue \leftarrow A[i]$

while $i + runlength \leq n - 1$ **and** $A[i + runlength] = runvalue$

$runlength \leftarrow runlength + 1$

if $runlength > modefrequency$

$modefrequency \leftarrow runlength$; $modevalue \leftarrow runvalue$

$i \leftarrow i + runlength$

return $modevalue$

BALANCED SEARCH TREES

A self-balancing search tree is any node-based search tree that automatically keeps its height small during arbitrary item insertions and deletions. Examples include

- AVL Trees
- Red-Black Trees
- Scapegoat Trees
- 2-3 Trees (not binary)
- B-Trees (not binary)

6.2.3

BALANCED TREES 459

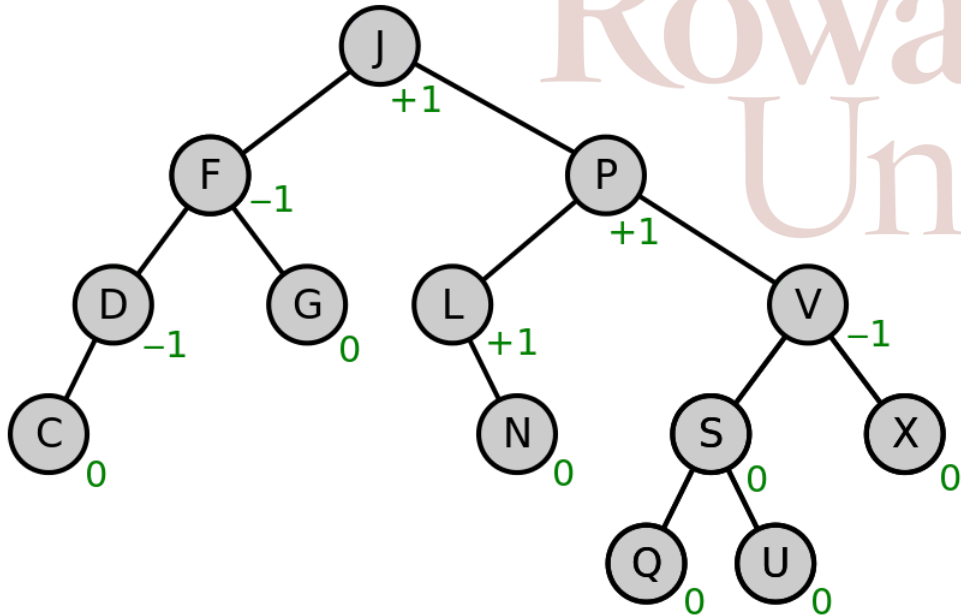
A very pretty solution to the problem of maintaining a good search tree was discovered in 1962 by two Russian mathematicians, G. M. Adelson-Velsky and E. M. Landis [*Doklady Akademii Nauk SSSR* **146** (1962), 263–266; English translation in *Soviet Math.* **3**, 1259–1263]. Their method requires only two extra bits per node, and it never uses more than $O(\log N)$ operations to search the tree or to insert an item. In fact, we shall see that their approach also leads to a general technique that is good for representing arbitrary *linear lists* of length N , so that each of the following operations can be done in only $O(\log N)$ units of time:

- i) Find an item having a given key.
- ii) Find the k th item, given k .
- iii) Insert an item at a specified place.
- iv) Delete a specified item.

AVL-TREES

An AVL tree (named after its two inventors Adel'son-Vel'skii and Landis) is a [self-balancing binary search tree](#) where the height of every node and that of its sibling differ by at most 1. AVL trees are **height-balanced**. Height can be calculated recursively; however, it is

generally faster for each node to keep track of its subtree height during insertions and deletions. While this is also done recursively, only the subtrees with a direct path from a new node to root need to be involved.



```
public class AVLTree<Key extends Comparable<Key>, Value> {

    public class Node<Key, Value> {
        Key key;
        Value value;
        int height;
        Node<Key, Value> left;
        Node<Key, Value> right;

        Node(Key key, Value value) {
            this.key = key;
            this.value = value;
            this.height = 0;
            this.left = null;
            this.right = null;
        }
    }

    private Node<Key, Value> root;
    private final Comparator<Key> comparator;
```



```
public AVLTree() {
    root = null;
    comparator = new Comparator<Key>() {
        @Override
        public int compare(Key o1, Key o2) {
            return o1.compareTo(o2);
        }
    };
}

public AVLTree(Comparator<Key> keyComparator) {
    root = null;
    comparator = keyComparator;
}

public int size() { return size(this.root); }

private int size(Node<Key, Value> node) {
    return node == null ? 0 : 1 + size(node.left) + size(node.right);
}
```

Now we can instantiate an AVL tree as follows:

```
AVLTree<Integer, String> T = new AVLTree<>();
```

Before we begin to fill our AVL tree, let's recognize that we already have a method for searching, because it is just BST search (with generic keys).



SEARCH

```
/**
 * <p>Non-recursive implementation of search for a BST (AVL Tree)</p>
 */
public Value search(Key key) throws FindException {
    Node<Key, Value> node = root;
    while (node != null) {
        int compareResult = comparator.compare(key, node.key);
        if (compareResult == 0) {
            break;
        }
        node = compareResult < 0 ? node.left : node.right;
    }
    if (node != null)
        return node.value;
    else
        throw new FindException("Key not found.");
}
```

INSERT

A node n of an AVL tree T is called ***balanced*** if the absolute value of the difference between the heights of its children is at most 1. Otherwise, it is called unbalanced.

An insertion in an AVL tree T begins as an **INSERT** operation for a standard binary search tree. This insertion may violate the height-balance property since the height of some subtrees may have increased by 1. Those subtrees are exactly the trees that are on the path from the new node to root.

```
public void insert(Key key, Value value) {
    root = insert(root, key, value);
}

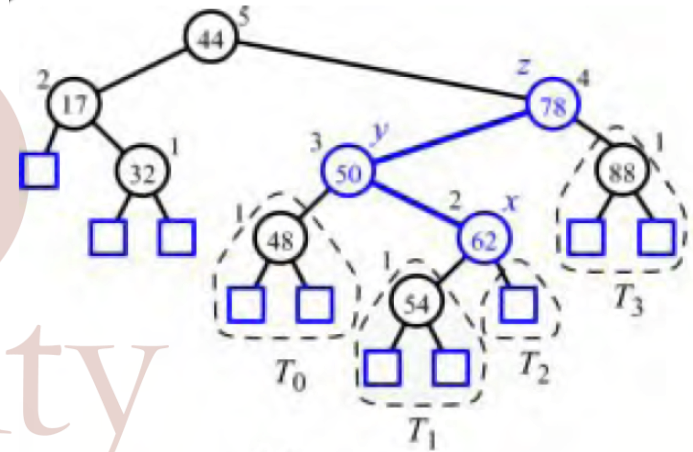
private Node<Key, Value> insert(Node<Key, Value> node, Key key, Value value) {
    if (node == null) {
        return new Node<Key, Value>(key, value);
    } else {
        int compareResult = comparator.compare(key, node.key);
        if (compareResult < 0) {
            node.left = insert(node.left, key, value);
        } else if (compareResult > 0) {
            node.right = insert(node.right, key, value);
        } else {
            // duplicate key
            // may throw runtime exception or quietly overwrite value
            node.value = value;
        }
    }
}

return rebalance(node);
}
```

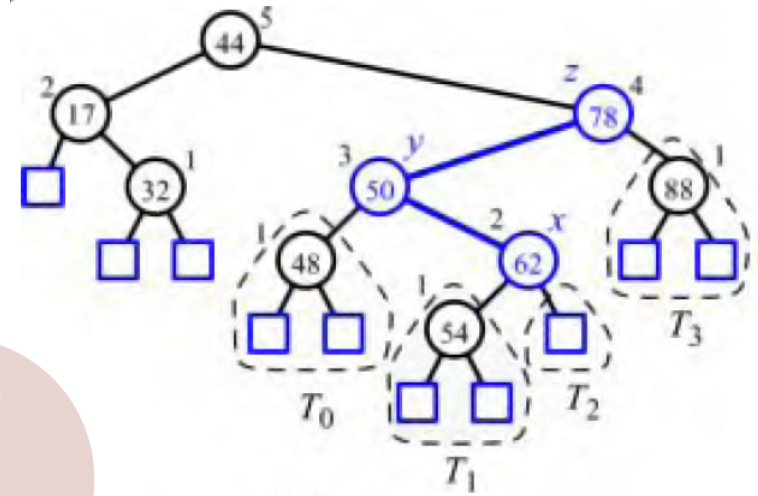
REBALANCING AN AVL TREE

We restore the balance of the nodes in the BST T using a search-and-repair strategy. Numbers next to nodes in subsequent pictures are the subtree heights + 1 (counting null children).

Let z be the first node encountered going up from newly inserted node n (key 54) toward the root of T such that z is unbalanced. Let y denote the child of z with higher height (note that y must be an ancestor of n because n added height to an imbalance of **2**). Let x be the child of y with higher height (there cannot be a tie and x must be an ancestor of n , possibly itself, again because of path length from n to z of at least **2**).



The repair is done with a tree rotation. We need an auxiliary function that checks for imbalance:



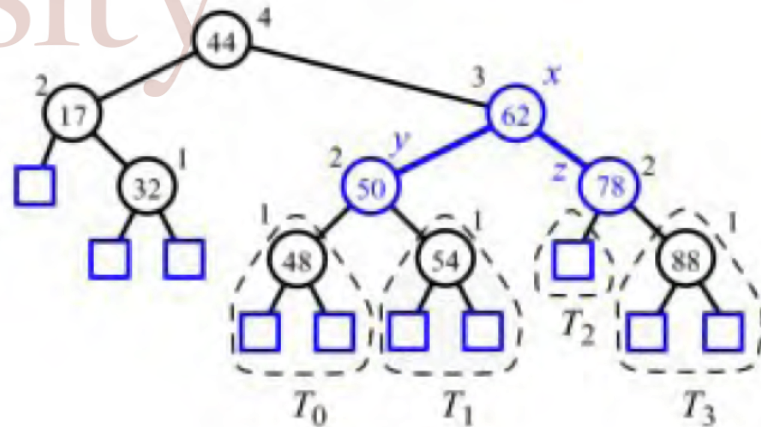
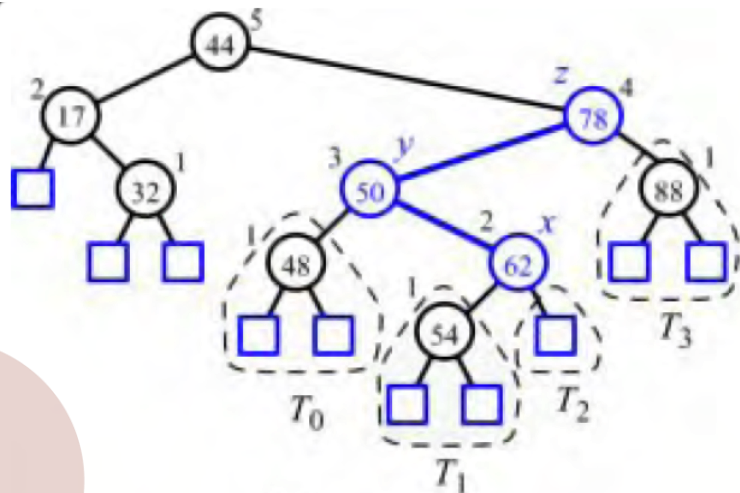
```
Balance(node z)
    return height(z.right) - height(z.left)
```




```

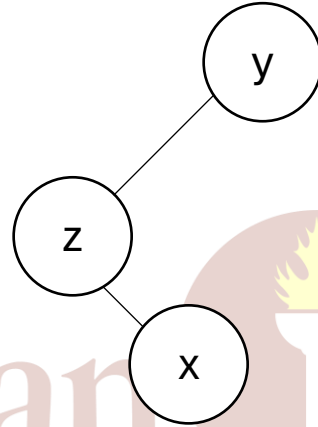
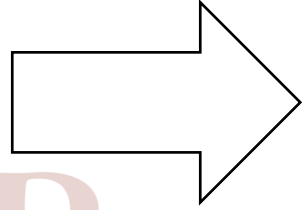
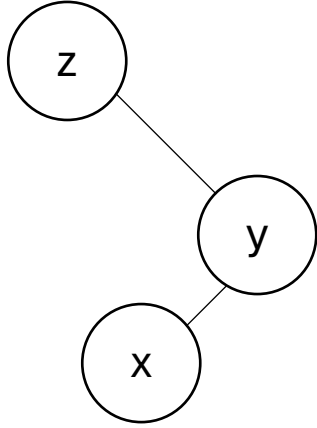
Rebalance(node z)
  if Balance(z)==2
    y = z.right
    if Balance(y)==2
      x = y.right
      z = rotateLeft(z)
    else
      x = y.left
      y = rotateRight(y)
      z = rotateLeft(z)
  else
    y = z.left
    if Balance(y)==-2
      x = y.left
      z = rotateRight(z)
    else
      x = y.right
      y = rotateLeft(y)
      z = rotateRight(z)

```



```
private Node<Key, Value> rebalance(Node<Key, Value> z) {
    updateHeight(z);
    int balance = getBalance(z);
    if (balance > 1) {
        if (height(z.right.right) > height(z.right.left)) {
            z = rotateLeft(z);
        } else {
            z.right = rotateRight(z.right);
            z = rotateLeft(z);
        }
    } else if (balance < -1) {
        if (height(z.left.left) > height(z.left.right)) {
            z = rotateRight(z);
        } else {
            z.left = rotateLeft(z.left);
            z = rotateRight(z);
        }
    }
    return z;
}
```

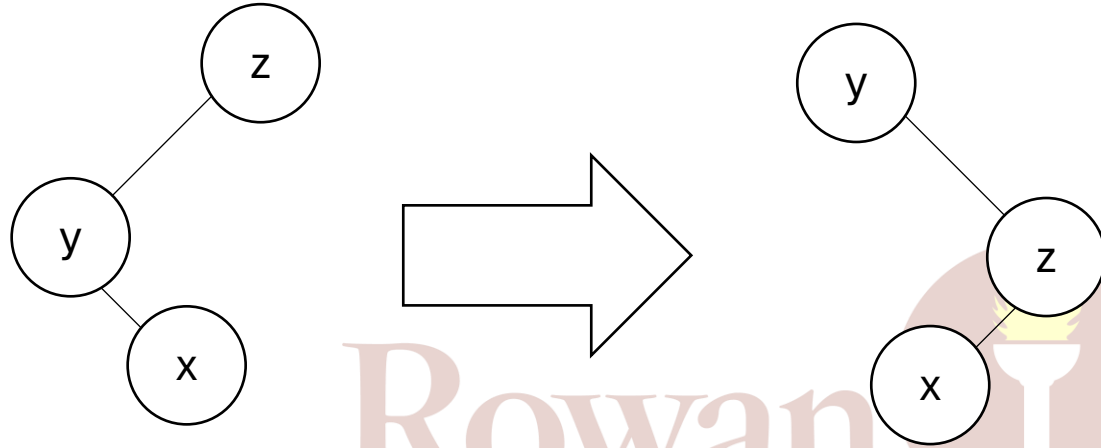
ROTATELEFT



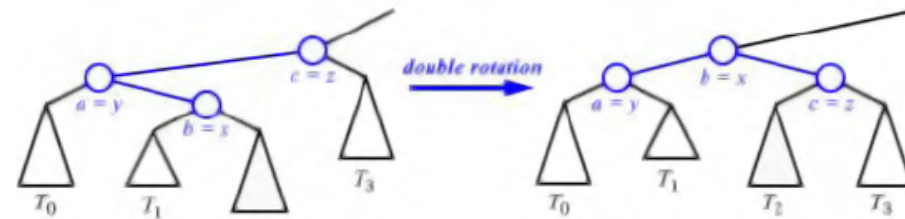
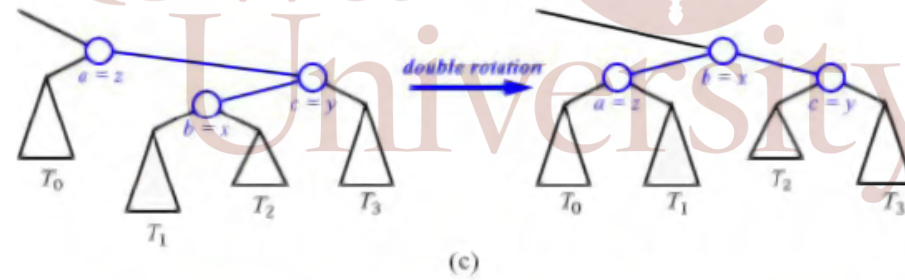
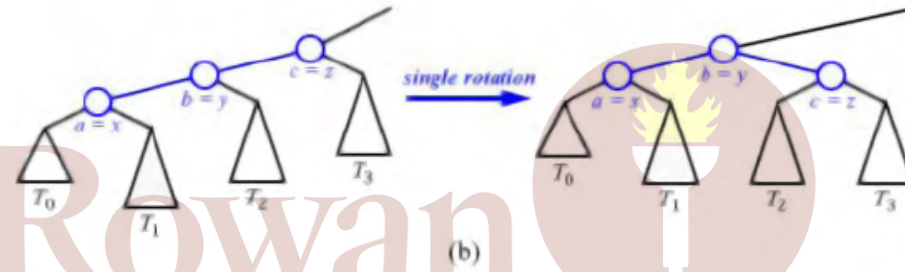
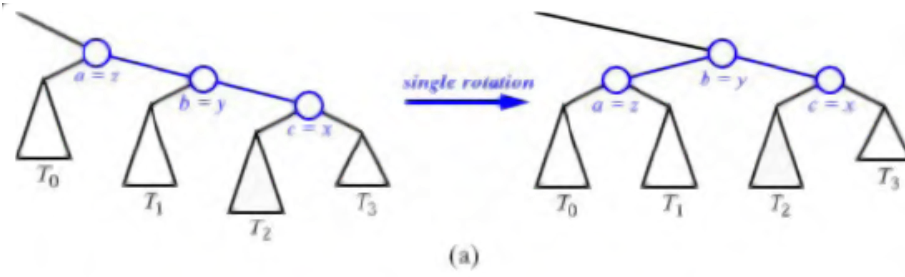
Rowan
University



ROTATERIGHT

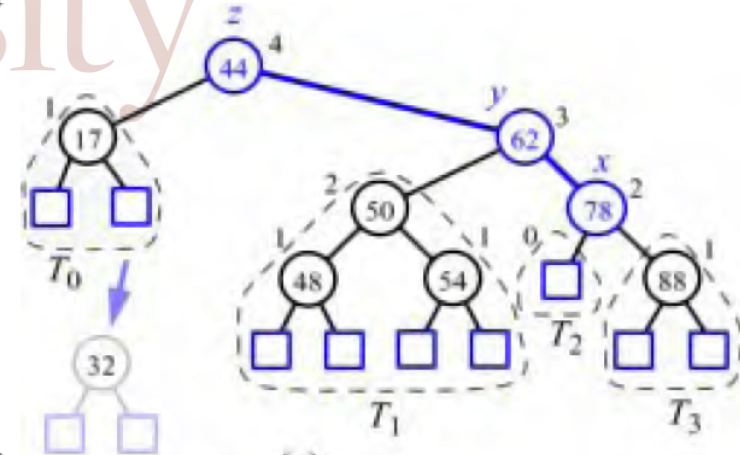


Rowan
University



DELETE

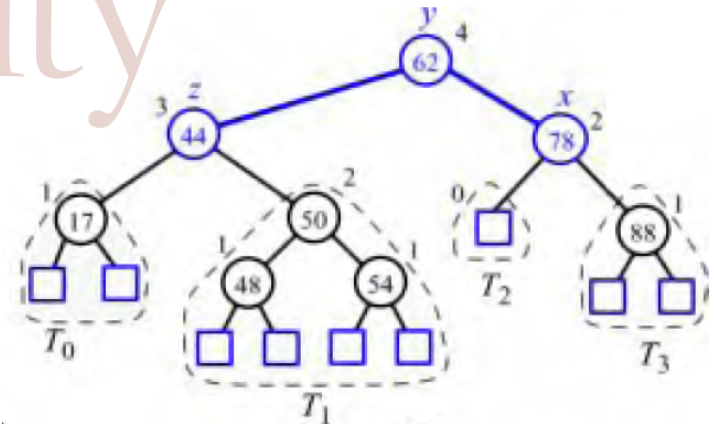
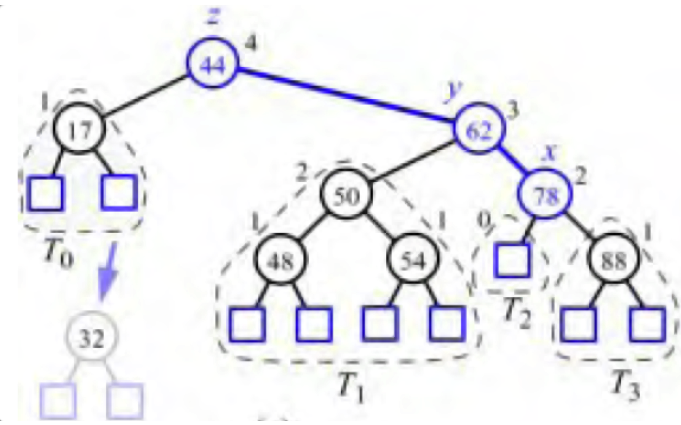
Removal of an element in an AVL tree also starts with the standard BST **DELETE**. Again, it may violate the height-balance property: after removing an internal node and elevating one of its children into its place, there may be an unbalanced node in T on the path from the parent node of the previously removed node to the root of T . However, there can be at most one such unbalanced node.



REBALANCING

Perform a **Rebalance(x)** operation. This restores the height-balance property **locally** at z . Rebalancing locally may reduce the height of the subtree, which may cause an ancestor to become unbalanced. Hence, after rebalancing z , continue to walk up T looking for unbalanced nodes until there are none or root has been rebalanced. In worst case, $O(\log(n))$ rebalancing operations are needed to restore balance **globally**. Per node a rotation takes $O(1)$ time.

```
public void delete(Key key) {  
    root = delete(root, key);  
}
```



```
private Node<Key, Value> delete(Node<Key, Value> node, Key key) {
    if (node == null) {
        return node;
    } else {
        int compareResult = comparator.compare(key, node.key);
        if (compareResult < 0) {
            node.left = delete(node.left, key);
        } else if (compareResult > 0) {
            node.right = delete(node.right, key);
        } else {
            if (node.left == null || node.right == null) {
                node = (node.left == null) ? node.right : node.left;
            } else {
                Node<Key, Value> leftMostChild = leftMostChild(node.right);
                node.key = leftMostChild.key;
                node.right = delete(node.right, node.key);
            }
        }
    }
    if (node != null) {
        node = rebalance(node);
    }
    return node;
}
```


PROPERTIES OF AVL TREES

- Find/Search/Get in an AVL tree is $O(\log(n))$.
- For every internal node of an AVL tree, the heights of the two children subtree differ by at most 1.
- A subtree of an AVL tree is itself an AVL tree.
- The height of an AVL tree storing n entries is $O(\log(n))$.
 - Can be proven with a recursion formula bottom up.

2-3 TREES

We set out to create guaranteed balanced search trees. The primary step to get the flexibility that we need to guarantee balance in search trees is to allow the nodes in our trees to hold more than one key. Specifically, referring to the nodes in a standard BST as 2-nodes (they hold two links and one key), we will now also allow 3-nodes, which hold three links and two keys. Both 2-nodes and 3-nodes have one link for each of the intervals subtended by its keys.

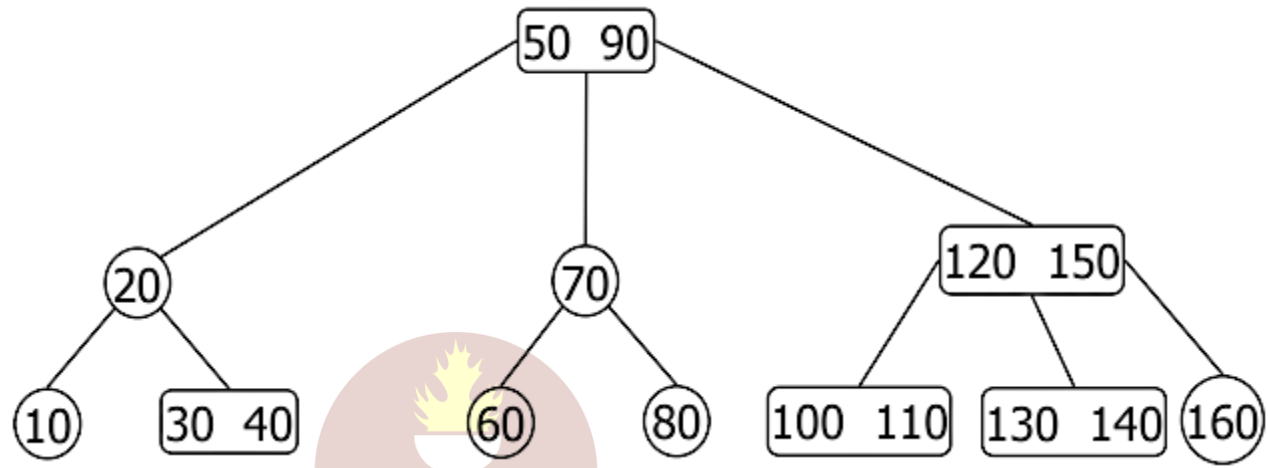
Unlike standard BSTs, which grow top down, **2-3 trees will grow up from the bottom**. Its balancing algorithm will rely on ***splits*** and ***merges*** of nodes (unlike Red-Black trees and AVL trees, which use ***rotations***).

A **2-3 Search Tree** (2-3 tree) is a tree that is either empty or

- a 2-node, with one key (and associated value) and two links,
 - a left link to a 2-3 search tree with smaller keys, and
 - a right link to a 2-3 search tree with larger keys
- a 3-node, with two keys (and associated values) and three links
 - a left link to a 2-3 search tree with smaller keys,
 - a middle link to a 2-3 search tree with keys between the node's keys, and
 - a right link to a 2-3 search tree with larger keys.

Note that links could be **null**!

A perfectly balanced 2-3 search tree is one whose leaves are all at the same distance from root:



- each interior node has either 2 or 3 children (possibly **null**)
- **all the leaves are at the same level** (as part of construction)

Each interior node contains one or two values:

- $L[v]$... largest value in the left subtree
- $M[v]$... largest values in the middle or second subtree

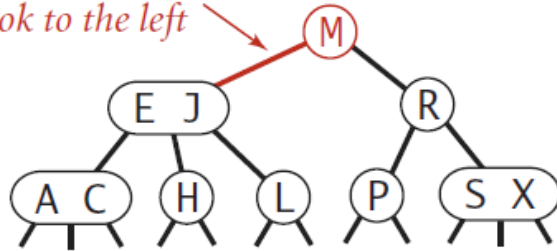
SEARCH

The search algorithm for keys in a 2-3 tree directly generalizes the search algorithm for BSTs. To determine whether a key is in the tree, we

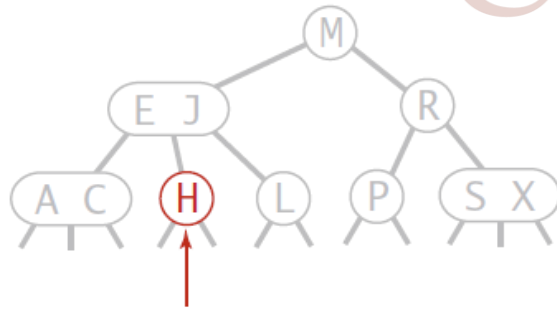
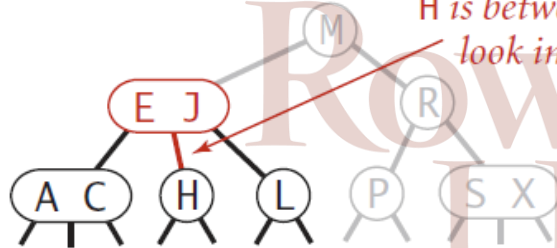
- compare it against the keys at the root. If it is equal to any of them, we have a search hit; otherwise,
- follow the link from root to the subtree corresponding to the interval of key values that could contain the search key.
 - if that link is null, we have a search miss; otherwise
 - recursively (or while loop) search in that subtree.

successful search for H

*H is less than M so
look to the left*



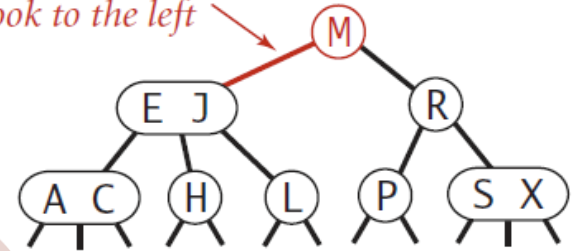
*H is between E and J so
look in the middle*



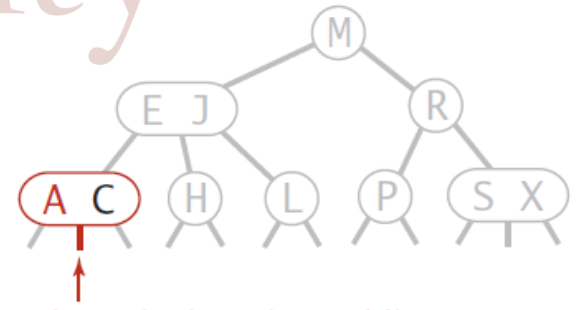
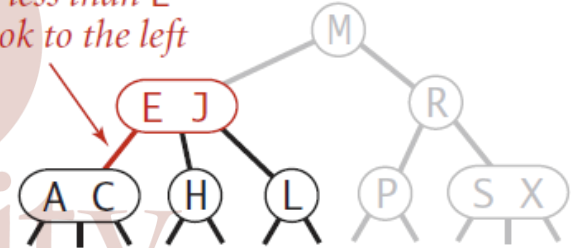
found H so return value (search hit)

unsuccessful search for B

*B is less than M so
look to the left*



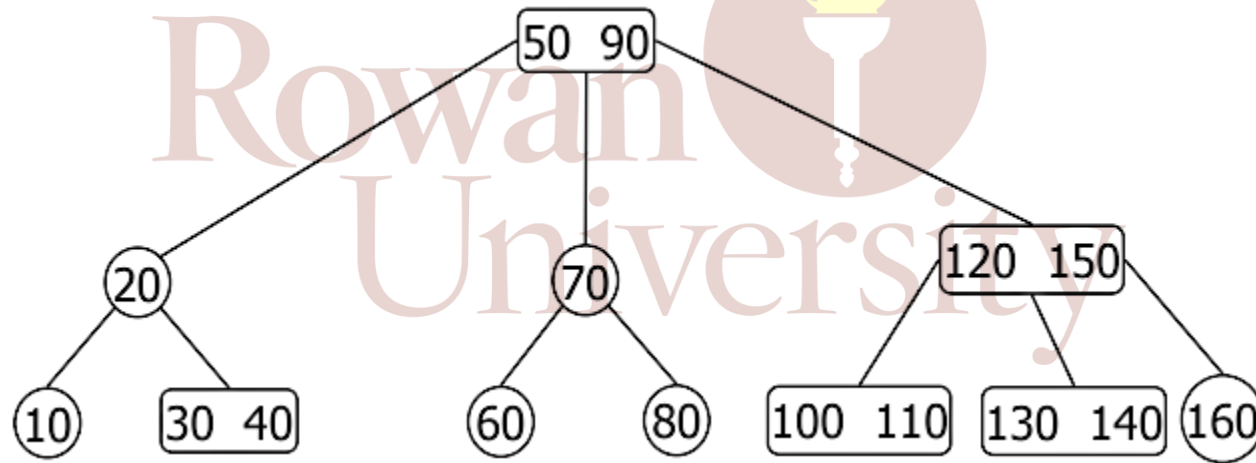
*B is less than E
so look to the left*



*B is between A and C so look in the middle
link is null so B is not in the tree (search miss)*

MIN AND MAX

Travel through the 2-3 tree to reach to the leftmost or the rightmost leaf. Cost is $O(\log(n))$. Alternatively, we can keep (and update) pointers to largest and smallest elements (cost $O(1)$).

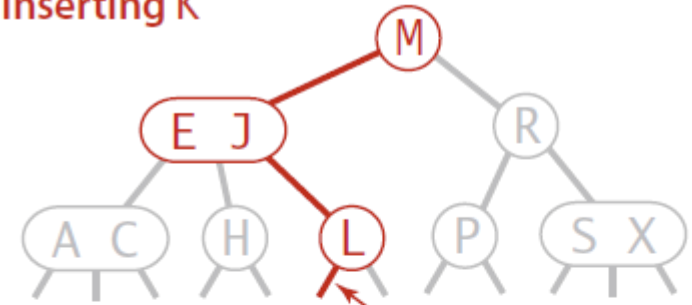


INSERT (2-NODE)

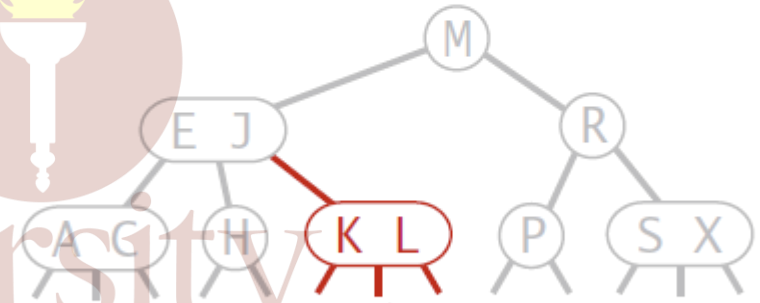
In general, to insert a new node in a 2-3 tree, we might do an unsuccessful search and then add a node at the bottom, as we did with BSTs. However, then the new tree would not remain perfectly balanced. We will need to consider cases!

If the search terminates at a 2-node, we can insert and turn it into a 3-node while still maintain perfect balance.

inserting K



search for K ends here



*replace 2-node with
new 3-node containing K*

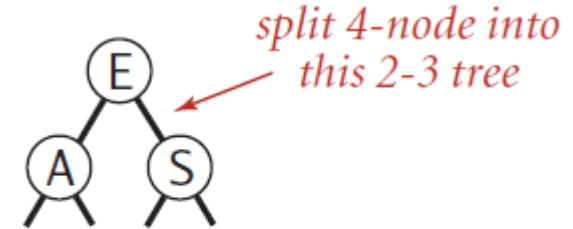
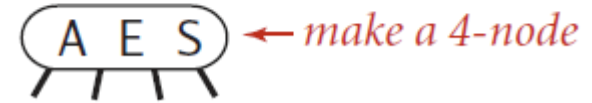
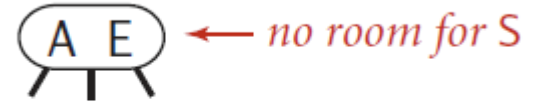
Insert into a 2-node

INSERT (TREE IS SINGLE 3-NODE)

Before we consider the general case, suppose we have a simplistic 2-3 tree consisting of just one 3-node (root). Such a tree has two keys, but no room for a new key in its one node.

- Temporarily create a 4-node
- Split 4-node into 2-3 tree with root and two 2-nodes

inserting S



Insert into a single 3-node

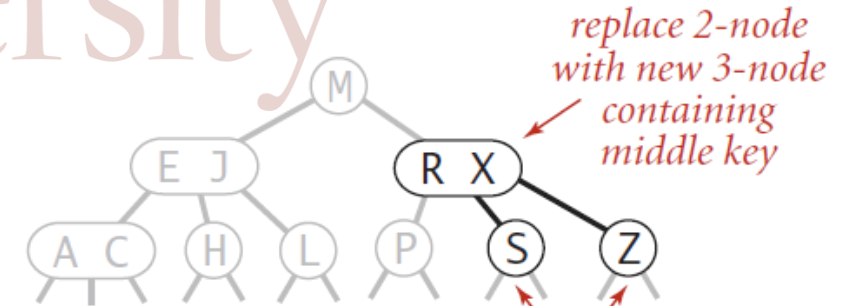
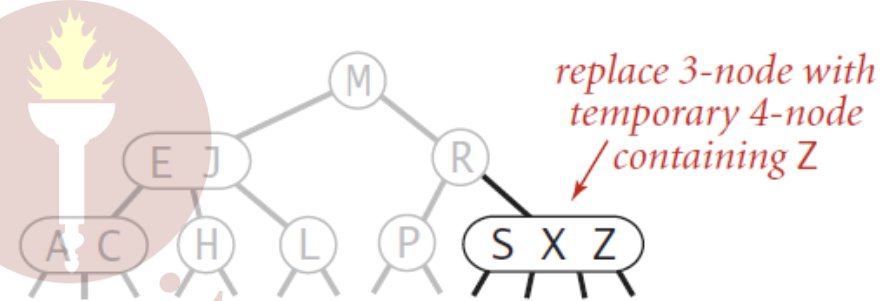
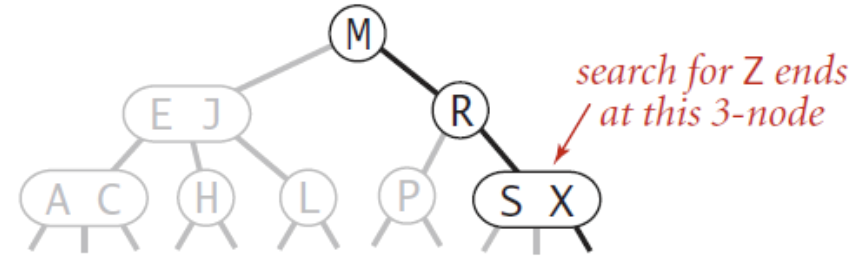
This works for this artificial example, but it **increases the height of the tree!**

INSERT (3-NODE WITH PARENT 2-NODE)

We can make room for the new key while maintaining balance in the tree: make a temporary 4-node and then split the 4-node by moving the middle key to the node's parent.

- Temporarily create 4-node
- Split 4-node by moving middle key to parent (at the correct position). Make left and right key of 4-node new 2-node children.

inserting Z



split 4-node into two 2-nodes
pass middle key to parent

We should notice a theme here: As long as there is some 2-node above the insertion, we should be able to make room for a new key in a 3-node by moving a key from the insertion 3-node up. Eventually this will fail when all nodes are 3-nodes along an insertion path, though.

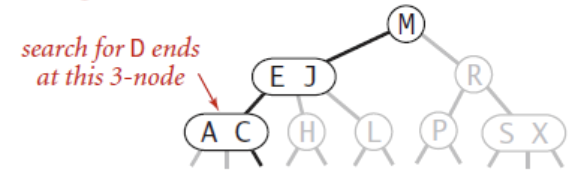


INSERT (3-NODE WITH PARENT 3-NODE)

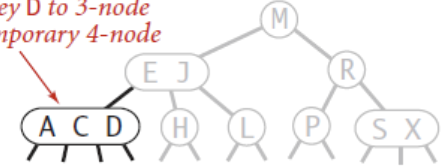
- Make a temporary 4-node at insertion
- Split insertion 4-node and add middle key to parent
 - Repeat recursively, if possible, until no splitting is necessary, or root is turned into a 4-node

If root is turned into a 4-node, we need to split root and increase the height of the tree.

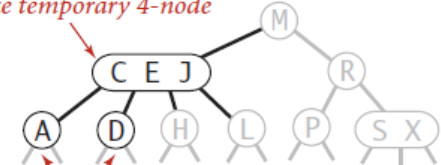
inserting D



add new key D to 3-node to make temporary 4-node

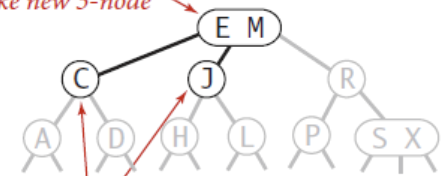


add middle key C to 3-node to make temporary 4-node



split 4-node into two 2-nodes pass middle key to parent

add middle key E to 2-node to make new 3-node



split 4-node into two 2-nodes pass middle key to parent

Insert into a 3-node whose parent is a 3-node

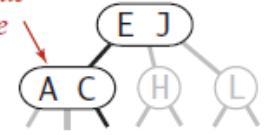
INSERT (SPLITTING THE ROOT)

If we have 3-nodes along the whole path from the insertion point to root, we end up with a temporary 4-node at root. We already looked at this case in our first attempt at 3-nodes.

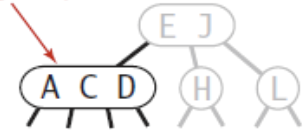
Now we have a theoretical procedure for inserts into a perfectly balanced 2-3 tree. These trees will maintain balance while growing and guarantee $O(\log(n))$ runtime. What about **Insert** runtime?

inserting D

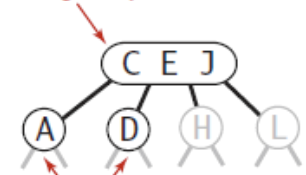
search for D ends
at this 3-node



add new key D to 3-node
to make temporary 4-node

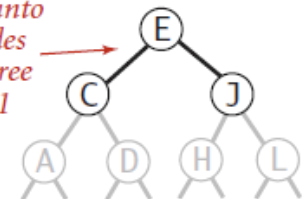


add middle key C to 3-node
to make temporary 4-node



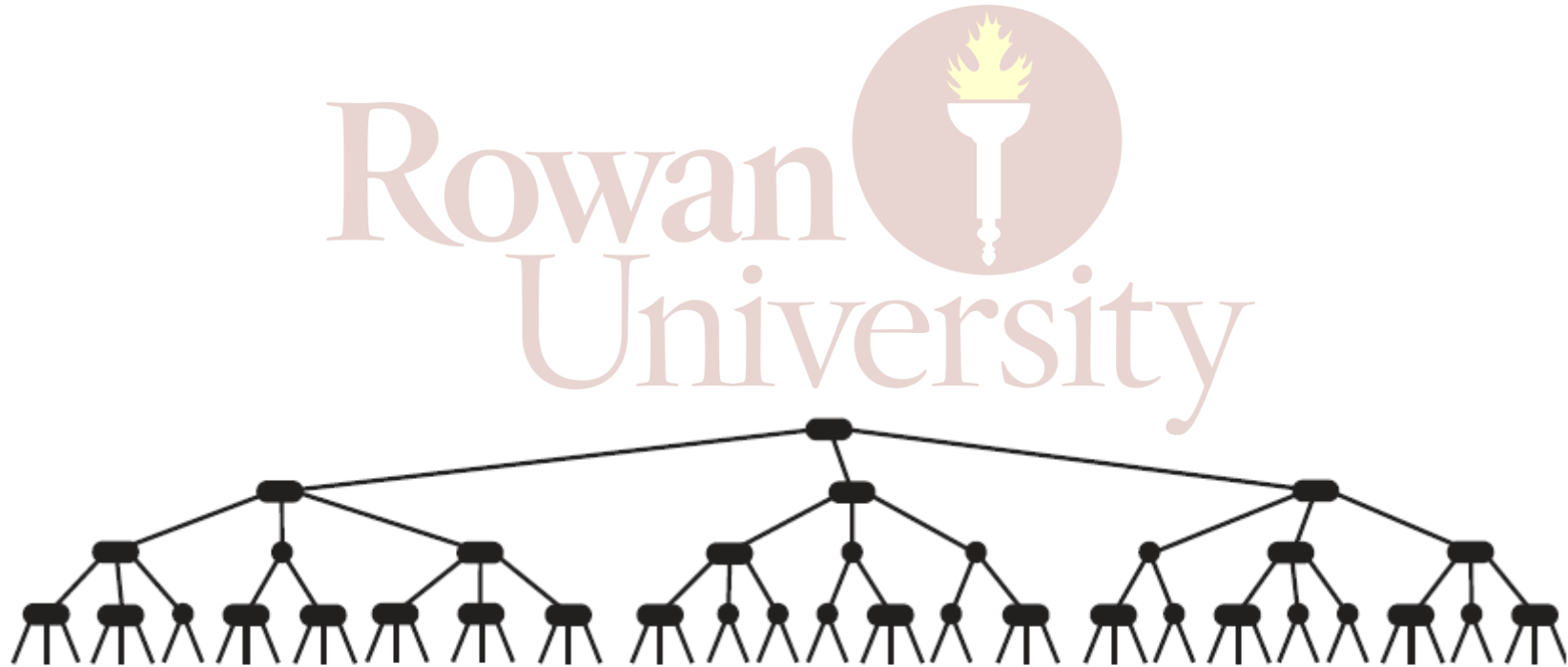
split 4-node into two 2-nodes
pass middle key to parent

split 4-node into
three 2-nodes
increasing tree
height by 1



Splitting the root

Example: We have seen in class that an ordered set inserted into a BST will create a linked list. Insert the ordered set of keys $\{1,2,3,4,5,6,7\}$ into an empty 2-3 tree.



Typical 2-3 tree built from random keys

DELMIN

Deletion of elements is more involved since we want to maintain balance. Let's consider deleting the minimum.

The minimum is the item we reach following left pointers from root. We can easily delete a key from a 3-node at the bottom of the tree, but not from a 2-node. Deleting the key from a 2-node leaves a node with no keys, and we cannot simply leave it as null since that violates the balance. What we could do *on the path from root down to the minimum* is to make sure every node visited is a 3-node or temporary 4-node by performing appropriate transformations to the subtree. This way, a 2-node leaf can borrow from its parent using a ***split***. If the split is not needed (because the minimum is actually in a 3-node), we can ***merge*** on our way back up.

At root, there are two possibilities for a 2-node:

- root is a 2-node and both children are 2-nodes
 - convert the three nodes to a 4-node
- root is a 2-node and at least one child is a 3-node
 - borrow from the right sibling if necessary to ensure that the left child of the root is not a 2-node

at the root



on the way down



at the bottom



Transformations for delete the minimum

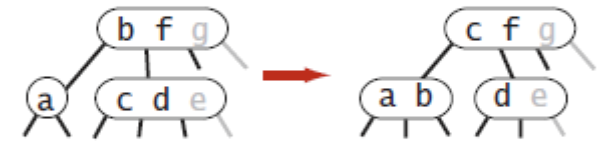
On the way down to the minimum, one of the following must hold:

- If the left child of the current node is not a 2-node, there is nothing to do.
- If the left child is a 2-node and its immediate sibling is not a 2-node, move a key from the sibling to the left child.
- If the left child and its immediate sibling are 2-nodes, then combine them with the smallest key in the parent to make a 4-node, changing the parent from a 3-node to a 2-node or from a 4-node to a 3-node.

at the root



on the way down



at the bottom

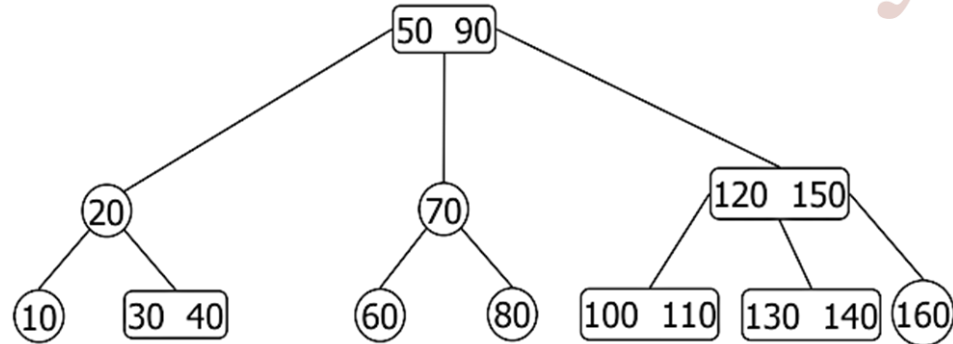


Transformations for delete the minimum

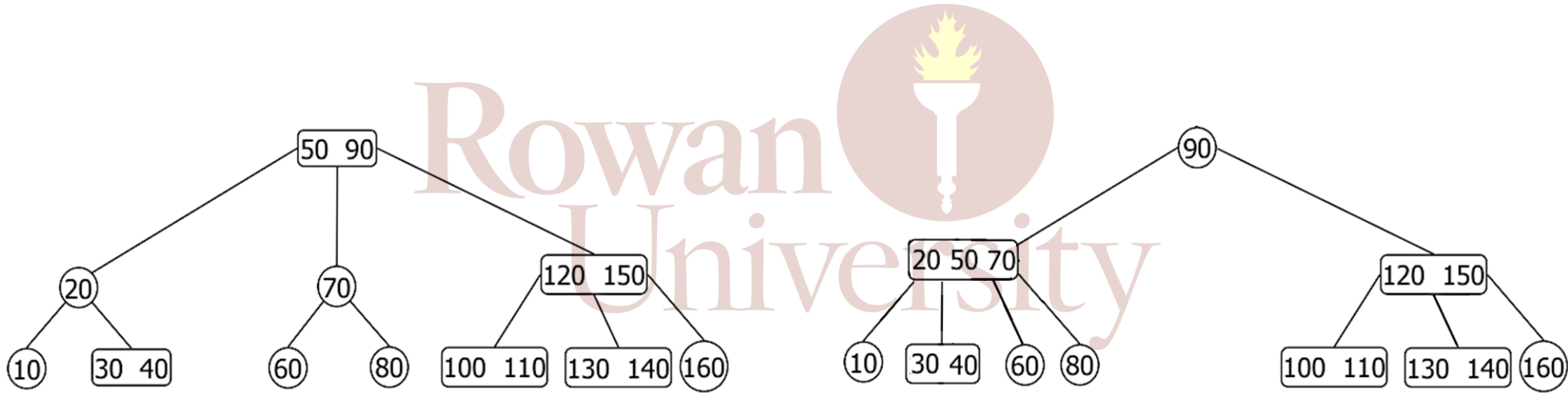
Following this process as we traverse left links to the bottom, we wind up on a 3-node or a 4-node with the smallest key, so we can just remove it, converting the 3-node to a 2-node or the 4-node to a 3-node. On the way up we split any unused temporary 4-nodes.



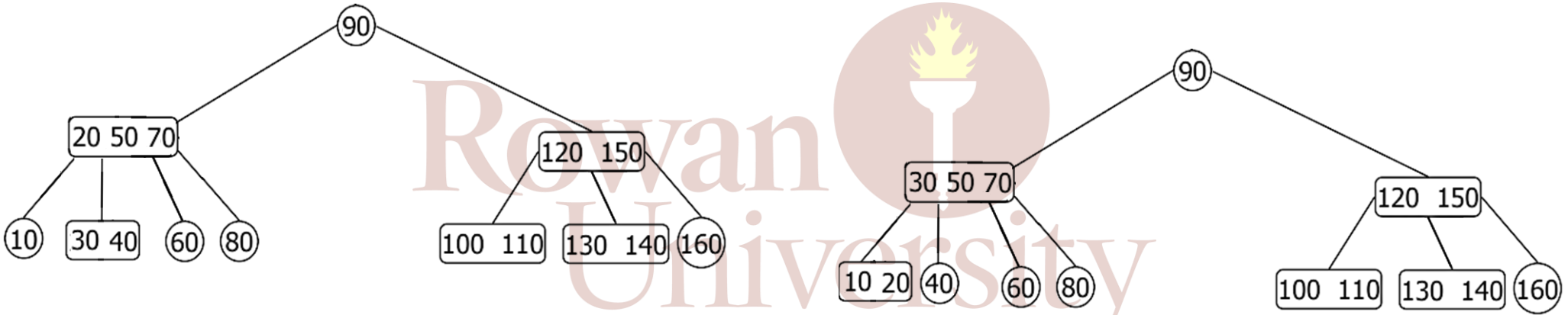
Example: Delete the minimum in the given 2-3 tree using the algorithm just outlined. Merge while moving from root to minimum, and split (if necessary) moving back from new minimum to root. Note that after finding the minimum, we can see that it is a 2-node. A 3-node would cause no issue.



Step 1: Left child of root is a 2-node, so we need to bubble down a key (data) to the left. From root (which here is a 3-node) to left child, bubble down left key and temporarily create a 4-node.



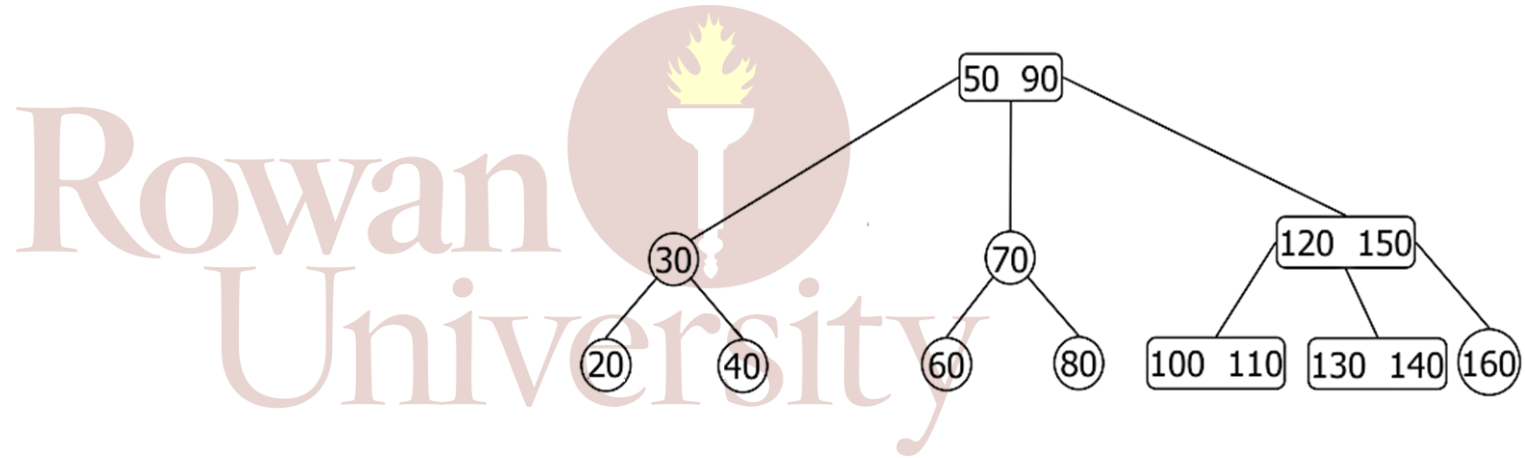
Step 2: From the current node (20, 50, 70), its left child is a 2-node. Bubble down key 20 and move up key 30.



Step 3: Delete minimum which is a 3-node.



Step 4: On the way from minimum towards root, parent of current node (20) is a 4-node. Split like we inserted a node. Middle key (50) bubbles up. Parent (90) becomes a 2-node with right sibling (70) going to the middle.



The same transformations along the search path just described for deleting the minimum are effective to ensure that the current node is not a 2-node during a search for any key. If the search key is at the bottom, we can just remove it. If the key is not at the bottom, then we have to exchange it with its successor as in regular BSTs. Then, since the current node is not a 2-node, we have reduced the problem to deleting the minimum in a subtree whose root is not a 2-node, and we can use the procedure just described for that subtree. After the deletion, we split any remaining 4-nodes on the search path on the way up the tree.

DELETE

1. Find the element to be deleted.
2. Delete the element
 - a. If the subtree is left with 2 leaves then
 - i. Update internal nodes. Done
 - b. If the subtree is left with only one leaf then
 - i. Check the sibling (left or right)
 - ii. If the sibling has only two children then remove the parent and combine the child with the children of the sibling. If again the tree is left with only one child then repeat the same process recursively until either obtained a normal 2-3 tree or reached the root. If reached the root then delete it. Update internal nodes as you travel up.
 - iii. If the sibling has three children then take rightmost/leftmost child of the sibling and add it to this node's parent.

INSERT RUNTIME ANALYSIS

Method	Average/Amortized	Worst case
Search	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$
Delete	$O(\log(n))$	$O(\log(n))$

- Find insertion/deletion position is $O(\log(n))$ due to balance.
- We need to split at most $O(\log(n))$ times since there are only $\log(n)$ levels and we do constant amount of work at each level.

Splitting a 4-node is a local transformation which preserves order and perfect balance. Moreover, these local transformations preserve the global properties that the tree is ordered and perfectly balanced. The height of the tree is between $\log_3(n)$ and $\log_2(n)$.

2-3 trees are usually not implemented using the operations we described since we would need to track two different types of nodes. The overhead of constantly testing what type of node we have might eat up all the speedup.



QUIZ TOPICS

- AVL tree and 2-3 tree facts
- Runtime for standard operations of AVL trees and 2-3 trees

